



Sprint & GitHub-Issue Roadmap

Goal:

Turn the three PDF exercises into a clean, test-driven Python project that can be released, documented, and continuously integrated.

Platform:

GitHub (Issues, Projects, Labels) – the same approach works for GitLab, Bitbucket, etc.

TL;DR: 5 sprints, 27 distinct issues (plus some epics & reusable labels).

Each sprint is 1-2 weeks long (adjust to your cadence).

0 Project Preparation (Sprint 0 – optional)

Issue	Title	Brief	Labels
#0	Create repo template	Set up empty repo, README, <code>.gitignore</code> , CI config (GitHub Actions or GitLab CI).	<code>init</code> , <code>repo</code> , <code>ci</code>
#1	Create GitHub Project board	Use <i>Project</i> → <i>Kanban</i> to host epics, stories, and tasks.	<code>project</code> , <code>planning</code>
#2	Define issue template	Custom “Feature” and “Bug” templates with fields: <i>User story</i> , <i>Acceptance criteria</i> , <i>Test cases</i> .	<code>template</code> , <code>documentation</code>
#3	Create label taxonomy	Base labels: <code>epic</code> , <code>story</code> , <code>task</code> , <code>bug</code> , <code>feature</code> , <code>good-first-issue</code> , <code>p0</code> , <code>p1</code> , <code>p2</code> , <code>documentation</code> , <code>test</code> , <code>ci</code> , <code>refactor</code> .	<code>labels</code>

⚠ **Tip:** Keep sprint boards *per sprint*. Move issues from “Backlog” → “Sprint 1” → “Done”.

1 Sprint 1 – Epic 1: Project Skeleton & Tests

Issue	Title	Description	Labels
#10	Create src directory	<code>mkdir src && touch src/__init__.py</code>	task
#11	Create tests directory	<code>mkdir tests && touch tests/__init__.py</code>	task
#12	Add tox + pytest	Configure <code>tox.ini</code> , <code>pytest.ini</code> for cross-environment testing.	setup, test, ci
#13	Add flake8 & black	Basic linting / formatting.	code quality
#14	Add README skeleton	Intro, license, installation, usage sections.	documentation
#15	Add LICENSE file (MIT)	Keep it short.	license

Outcome: A fully test-ready, lint-clean repo, ready to drop in code.

2 Sprint 2 – Epic 2: Exercise 1 – Palindrome Detection

Issue	Title	Acceptance Criteria
#20	Feature: Palindrome detection class	Given a string, <i>when</i> <code>is_palindrome()</code> is called, <i>then</i> returns <code>True</code> only if reversible.

Issue	Title	Acceptance Criteria
#21	Add CLI wrapper	<code>python -m palindrome_cli <string></code> prints result.
#22	Unit tests	5 tests: simple palindrome, case-insensitive, spaces, punctuation, non-palindrome.
#23	Docstring & README	Explain usage, example – <code>palindrome.py</code> .
#24	Good-first-issue label	Mark this for newbies.

Optional: Add a small `README.md` snippet for *Exercise 1*.

3 Sprint 3 – Epic 3: Exercise 2 – Password Strength Checker

Issue	Title	Acceptance Criteria
#30	Feature: Password strength evaluator	Return score (0-5) based on criteria, with explanation.
#31	Add CLI wrapper	<code>python -m passwd_cli <pwd></code> prints result & score.
#32	Unit tests	6 tests: missing digit, missing uppercase, missing special, all ok, short length, complex.
#33	Docstring & README	Sample usage, config options.
#34	Refactor	Move common regex to <code>common.py</code> .
#35	Good-first-issue label	New contributors may start here.

 **Optional bonus:** Add a web form via Flask or FastAPI for demo.

4 Sprint 4 – Epic 4: Exercise 3 – Text Analysis

Issue	Title	Acceptance Criteria
#40	Feature: Text analysis engine	Calculates <i>words</i> , <i>sentences</i> , <i>avg word length</i> , <i>avg sentence length</i> .
#41	Add CLI wrapper	<code>python -m analysis_cli <text></code> prints structured summary.
#42	Unit tests	4 tests covering punctuation, edge-cases, multi-sentence, single word.
#43	Docstring & README	Show example input & output.
#44	Documentation style guide	Maintain a section in README.
#45	Good-first-issue label	Candidate for newcomers.



Stretch goal: Use NLTK or spaCy for robust sentence tokenization.

5 Sprint 5 – Epic 5: Polish & Release

Issue	Title	Description
#50	Update README	Consolidate all exercises, installation, CI results, contributing.
#51	Add badges	Build, tests, license, coverage.
#52	Add version command	<code>python -m version</code> prints <code>vX.Y.Z</code> .
#53	Add changelog	<code>CHANGELOG.md</code> with semantic versioning.
#54	Write CONTRIBUTING guide	Branching, PR template.
#55	Publish on PyPI (optional)	Create <code>setup.cfg</code> , <code>pyproject.toml</code> .
#56	Deploy demo (optional)	Deploy to Netlify/Vercel/AWS for web demos.

Issue	Title	Description
#57	Retrospective	Document lessons & future improvements.



Sample GitHub Issue Template (for features)

****User story****

As a user, I want to see if a string is a palindrome so that I can do

****Acceptance criteria****

- The ``is_palindrome`` function returns ``True`` for ``"madam"`` and ``"Racecar"``.
- The function ignores whitespace and punctuation.
- A CLI command prints ``Yes, it's a palindrome!`` when appropriate.

****Test cases****

Input	Expected Output
madam	Yes, it's a palindrome!
hello	No, it's not a palindrome.
racecar	Yes, it's a palindrome!



How to Add an Issue to a Sprint

1. **Create the issue** – fill out the template.
2. **Assign labels** – e.g., `epic`, `story`, `p0`.
3. **Move to board** – drag to “Sprint X” column.
4. **Assign to team member** or mark as `good-first-issue`.



Final Checklist

1. All three exercises have a module, CLI wrapper, unit tests, and docs.
2. CI runs on every push (tests + lint).
3. README includes clear installation, usage, and contribution guides.

4. Repo uses semantic versioning & changelog.
 5. Labels and issues follow the defined taxonomy.
-

Ready to Sprint?

Clone the template repo, fork it, and start moving issues through the boards.

Happy coding!